

A Technical Introduction to Q-learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Section 1

History Q-learning was introduced by Chris Watkins in 1989. A convergence proof was presented by Watkins and Peter Dayan in 1992, building on Watkins' doctoral dissertation, Learning from Delayed Rewards. Eight years earlier in 1981, the same problem, under the name of "Delayed reinforcement learning," was solved by Bozinovski's Crossbar Adaptive Array (CAA). The memory matrix $W = [w(a, s)]$ was the same as the eight years later Q-table of Q-learning. The architecture introduced the term "state evaluation" in reinforcement learning. The crossbar learning algorithm, written in mathematical pseudocode in the paper, in each iteration performs the following computation:

Section 2

Function approximation Q-learning can be combined with function approximation. This makes it possible to apply the algorithm to larger problems, even when the state space is continuous. One solution is to use an (adapted) artificial neural network as a function approximator. Another possibility is to integrate Fuzzy Rule Interpolation (FRI) and use sparse fuzzy rule-bases instead of discrete Q-tables or ANNs, which has the advantage of being a human-readable knowledge representation form. Function approximation may speed up learning in finite problems, due to the fact that the algorithm can generalize earlier experiences to previously unseen states.

Section 3

where R_{t+1} is the reward received when moving from the state S_t to the state S_{t+1} , and α is the learning rate ($0 < \alpha \leq 1$). Note that $Q^{\text{new}}(S_t, A_t)$ is the sum of three terms:

Section 4

Others Delayed Q-learning is an alternative implementation of the online Q-learning algorithm, with probably approximately correct (PAC) learning. Greedy GQ is a variant of Q-learning to use in combination with (linear) function approximation. The advantage of Greedy GQ is that convergence is guaranteed even when function approximation is used to estimate the action values.

Distributional Q-learning is a variant of Q-learning which seeks to model the distribution of returns rather than the expected return of each action. It has been observed to facilitate estimate by deep neural networks and can enable alternative control methods, such as risk-sensitive control.

Section 5

Learning rate The learning rate or step size determines to what extent newly acquired information overrides old information. A factor of 0 makes the agent learn nothing (exclusively exploiting prior knowledge), while a factor of 1 makes the agent consider only the most recent information (ignoring prior knowledge to explore possibilities). In fully deterministic environments, a learning rate of $\alpha_t = 1$ is optimal. When the problem is stochastic, the algorithm converges under some technical conditions on the learning rate that require it to decrease to zero. In practice, often a constant learning rate is used, such as $\alpha_t = 0.1$ for all t .

Section 6

$$Q_{t+1}^B(s_t, a_t) = Q_t^B(s_t, a_t) + \alpha_t (r_t + \gamma Q_t^A(s_{t+1}, \arg \max_a Q_t^B(s_{t+1}, a)) - Q_t^B(s_t, a_t))$$

$$Q_{t+1}^B(s_t, a_t) = Q_t^B(s_t, a_t) + \alpha_t (r_t + \gamma Q_t^A(s_{t+1}, \arg \max_a Q_t^B(s_{t+1}, a)) - Q_t^B(s_t, a_t))$$

Section 7

5 second wait time + 0 second fight time Through exploration, despite the initial (patient) action resulting in a larger cost (or negative reward) than in the forceful strategy, the overall cost is lower, thus revealing a more rewarding strategy.

Section 8

Multi-agent learning Q-learning has been proposed in the multi-agent setting (see Section 4.1.2 in). One approach consists in pretending the environment is passive. Littman proposes the minimax Q learning algorithm.

Section 9

Discount factor The discount factor γ determines the importance of future rewards. A factor of 0 will make the agent "myopic" (or short-sighted) by only considering current rewards, i.e. r_t (in the update rule above), while a factor approaching 1 will make it strive for a long-term high reward. If the discount factor meets or exceeds 1, the action values may diverge. For $\gamma = 1$, without a terminal state, or if the agent never reaches one, all environment histories become infinitely long, and utilities with additive, undiscounted rewards generally become

infinite. Even with a discount factor only slightly lower than 1, Q-function learning leads to propagation of errors and instabilities when the value function is approximated with an artificial neural network. In that case, starting with a lower discount factor and increasing it towards its final value accelerates learning.

Section 10

Implementation Q-learning at its simplest stores data in tables. This approach falters with increasing numbers of states/actions since the likelihood of the agent visiting a particular state and performing a particular action is increasingly small.

Section 11

Double Q-learning Because the future maximum approximated action value in Q-learning is evaluated using the same Q function as in current action selection policy, in noisy environments Q-learning can sometimes overestimate the action values, slowing the learning. A variant called Double Q-learning was proposed to correct this. Double Q-learning is an off-policy reinforcement learning algorithm, where a different policy is used for value evaluation than what is used to select the next action. In practice, two separate value functions Q^A and Q^B are trained in a mutually symmetric fashion using separate experiences. The double Q-learning update step is then as follows:

Section 12

$\alpha \gamma \max_a Q(S_{t+1}, a)$: the maximum reward that can be obtained from state S_{t+1} (weighted by learning rate and discount factor) An episode of the algorithm ends when state S_{t+1} is a final or terminal state. However, Q-learning can also learn in non-episodic tasks (as a result of the property of convergent infinite series). If the discount factor is lower than 1, the action values are finite even if the problem can contain infinite loops or paths. For all final states s_f , $Q(s_f, a)$ is never updated, but is set to the reward value r observed for state s_f . In most cases, $Q(s_f, a)$ can be taken to equal zero.